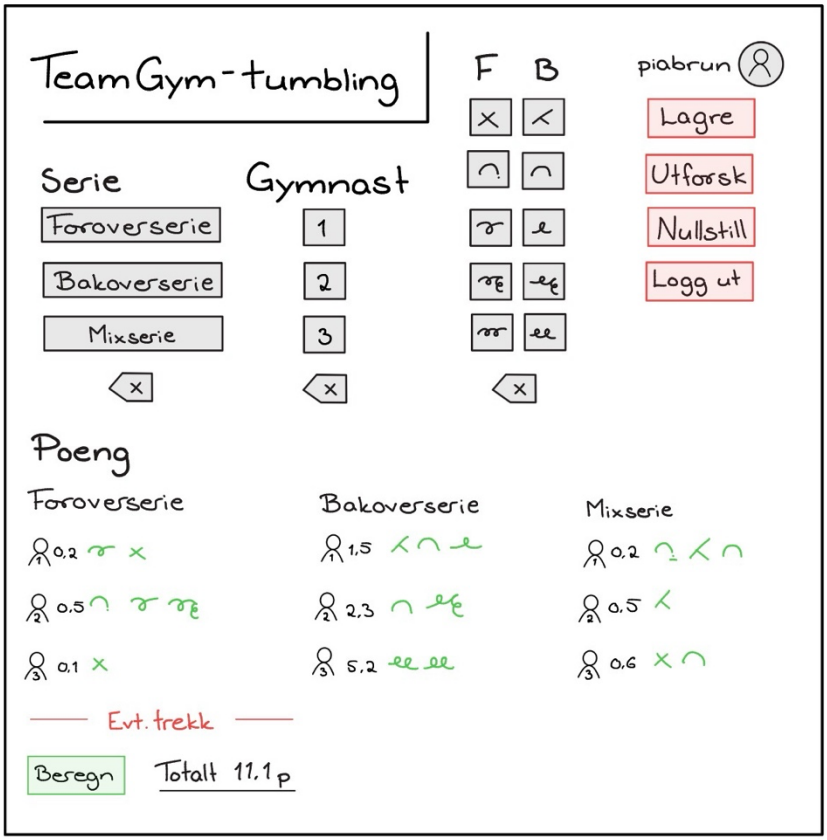
Prosjektoppgave

TEAMGYM-TUMBLING

Pia Brun

Ingeniørvitenskap & IKT



Idé

Resultat

Innholdsfortegnelse

Dokumentasjon **3**

Beskrivelse av appen *3*

Klassediagram *3*

Spørsmål *4*

Dekket pensum 4

Ikke dekket pensum 4

Model-View-Controller-prinsippet 5

Testing 5

KI Deklarasjon *6*

Punkter i deklarasjonen 6

Egne tanker og refleksjoner 6

Dokumentasjon

Beskrivelse av appen

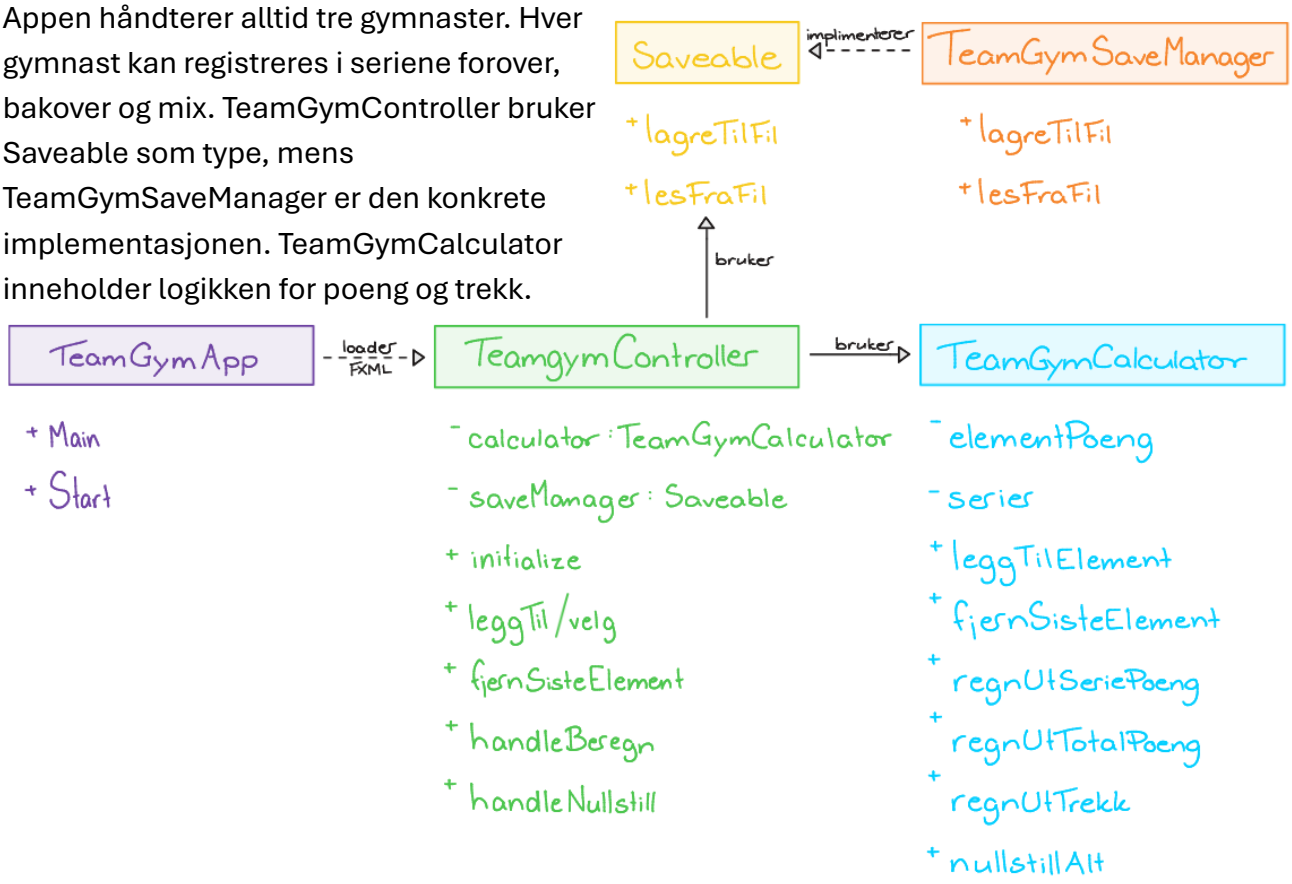
JavaFX-appen TeamGym-Tumbling hjelper brukere med å registrere tumblingserier for tre gymnaster i TeamGym. Appens hovedmål er å gjøre det enklere for trenere å holde oversikt over hvilke elementer hver gymnast utfører i forover-, bakover- og mixserie, samt beregne poeng og komposisjonstrekk automatisk ut fra faste regler.

Brukeren velger henholdsvis serie, gymnast og elementer ved hjelp av knapper i grensesnittet, dette vises fortløpende sammen med serieskår for hver enkelt gymnast, og kan regne ut både seriepoeng, trekk og total poengsum. Likeså kan brukeren fjerne siste element, nullstille hele registreringen og lagre eller lese inn data fra fil slik at en får opp prosjektet etter at appen er lukket.

Programmet er bygget etter Model-View-Controller-prinsippet. Brukergrensesnitteter laget i FXML, kontrolleren håndterer knappetrykk og oppdatering av skjermen, mens logikken for poengberegning og trekkregler ligger i egne modellklasser. Dette gjør koden oversiktlig og enklere å teste og utvikle.

Klassediagram

Appen håndterer alltid tre gymnaster. Hver gymnast kan registreres i seriene forover, bakover og mix. TeamGymController bruker Saveable som type, mens TeamGymSaveManager er den konkrete implementasjonen. TeamGymCalculator inneholder logikken for poeng og trekk.



Spørsmål

Dekket pensum

Appen bruker klasser og objekter for å dele opp ansvaret i programmet:

TeamGymApp starter appen
TeamGymCalculator har ansvar for logikk
TeamGymController håndterer brukerinput og koblingen til GUI
TeamGymSaveManager håndterer filbehandling

Dette viser hvordan programmet er delt opp i flere samarbeidende klasser med ulike ansvarsområder. Med dette tydelige skille mellom modell, view og controller kan man også observere at prosjektet dekker MVC-prinsippet. Brukergrensesnittet er laget i FXML, logikken i underliggende klasser mens kontrolleren binder de sammen.

Prosjektet dekker også innkapsling, ettersom at dataene i TeamGymCalculator er private og kan kun endres gjennom metoder som leggTilElement, fjernSisteElement og nullstillingsmetodene som gjør at klassen selv kontrollerer tilstanden sin. I tillegg brukes delegering, fordi kontrolleren ikke selv utfører beregningen og filbehandlingen, men sender den heller videre til TeamGymCalculator og TeamGymSaveManager. Prosjektet dekker også interface, ved at TeamGymSaveManager implementerer Saveable som viser bruk av kontrakter mellom interface og klasse.

Videre dekker prosjektet samlinger og datastrukturer, spesielt gjennom bruk av Map, Set, List og ArrayList. Disse brukes for å organisere serier, gymnaster og elementer. Prosjektet dekker også filbehandling, ettersom at appen kan lagre og lese inn tilstand fra tekstfila teamgym_lagring.txt, og testing, siden det er laget JUnit-tester for den sentrale funksjonaliteten som poengberegning, trekkregler, nullstilling og lagring.

Ikke dekket pensum

Prosjektet bruker ikke arv i noe stor grad. Det kunne vært brukt dersom appen skal utvides videre. Eksempelvis kunne man hatt en felles superklasse for ulike typer elementer eller serier, og så laget underklasser med spesialisert oppførsel. Dette er relevant ettersom at det er forskjellige regler for rekrutt, junior og senior.

En annen del av pensum som kunne vært brukt mer er validering og exceptions. Selv om appen har noe enkel feilhåndtering, kunne det vært forbedret ved å validere input tydeligere og kaste egne exceptions ved feil filformat og ugyldige serier. Per nå er det for eksempel ingenting som stopper brukeren fra å velge henholdsvis «salto», «backflipp», «hjul» selv om dette ikke ville fungert i praksis. Dette ville gjort løsningen mer robust og mer sikker behandling av tilstand.

Model-View-Controller-prinsippet

Koden er startet ut fra eksempel prosjektet som delvis fulgte prinsippet fra før. View-delen er definert i TeamGym.fxml, som beskriver brukergrensesnittet med knapper, labels og en meny. Controller-delen er TeamGymControler, som reagerer på brukerhandlinger som valg av serie, gymnast og elementer, samt lagring. Model-delen består hovedsakelig av TeamGymcalCalculator, som holder på dataene og poeng- og trekkberegninger, samt TeamGymSaveManager, som håndterer lagring og innlesing fra fil.

En styrke i løsningen er at selve beregningene ikke ligger i controlleren. Kontrolleren delegerer i stedet videre til modellklassene. Dette gjør løsningen mer ryddig og testbar. Samtidig er det noen svakheter. TeamGymController inneholder en del ansvar knyttet til visningslogikk, markering av knapper og tilbakemeldinger til brukeren. Det er naturlig at noe av dette ligger i controlleren, men klassen er blitt relativt stor, og kunne med fordel vært delt opp eller fått noen hjelpeklasser dersom prosjektet skal utvides.

En annen svakhet er at TeamGymCalculator både holder på data og inneholder mye regelverk for poeng og trekk. Dette fungerer i en liten app, men i en større løsning kunne det vært nyttig å skille data og regler enda tydeligere.

Testing

Det ble prioritert å teste den funksjonaliteten som er viktigst for a appen skal fungere riktig. Det ble derfor laget tester for TeamGymCalculator, siden denne klassen inneholder den sentrale logikken i programmet. Her er det testet at elementer kan fjernes og legges til, at poeng for en serie blir riktig utregnet, at serier kan nullstilles, at like serier blir oppdaget riktig, at trekk blir regnet ut riktig i ulike situasjoner, og at total poengsum blir riktig.

I tillegg ble det laget egne tester for TeamGymSaveManager, siden prosjektbeskrivelsen krevde dette. Her ble det testet at data faktisk ble skrevet til fil, og at data kan leses tilbake inn i etter appen ble lukket.

Det er bevist ikke testet direkte på TeamGymController eller TeamGymApp, ettersom prosjektbeskrivelsen ikke krevde dette av klassene med lite logikk. Fokus ble derfor lagt på modell- og lagringsklassene, der feil ville hatt størst konsekvenser for funksjonaliteten i applikasjonen.

Testene er valgt for å dekke både normal bruk og viktige tilfeller som kan føre til feil i beregningene. Samtidig er det ikke testet absolutt alle deler av koden, særlig ikke GUI-relatert oppførsel og visuelle oppdateringer. Dette ble nedprioritert fordi den viktigste funksjonaliteten ligger i beregningene og lagringen.

KI Deklarasjon

Punkter i deklarasjonen

Jeg måtte ikke bruke KI for å få til prosjektet, men det var ekstremt tidsbesparende og nyttig som støtteverktøy. Jeg brukte KI for å få forklaringer på begreper, hjelp til å forstå feilmeldinger, og forslag til hvordan jeg kunne strukturere deler av koden og testene. KI gjorde det lettere å komme videre når jeg stod fast, men prosjektet kunne i prinsippet blitt gjennomført uten.

Jeg har i hovedsak brukt OpenAI sin ChatGPT Thinking 5.4, hvor jeg har avskrudd «forbedre modellen for alle» med tanke på at jeg ikke ønsker at mitt input skal kunne bli brukt i andre sammenhenger enn med meg. Jeg har brukt ChatGPT i en ganske lang periode og føler meg trygg på denne språkmodellen. Til noe mindre spørsmål har jeg delvis brukt Copilot som ifølge seg selv blir kontinuerlig forbedret og oppdatert. Jeg vet det finnes flere modeller tilgjengelig, men per nå har jeg ikke prioritert tid til å utforske dette. Dersom det kommer nyheter på andre språkmodellers fremskritt skal jeg utforske det når den tid kommer.

Jeg ble litt nysgjerrig på hva som egentlig er tilgjengelig av retningslinjer og så at NTNU tilbyr to språkmodeller, GPT NTNU som kun er den nyeste varianten som var tilgjengelig i 2024.

Jeg brukte KI i moderat grad. Jeg spurte om hjelp til debugging, forklaring av feil, forslag til tester og hvordan enkelte deler av koden kunne forbedres. KI skrev forslag til kodeeksempler og testkode som jeg deretter måtte forstå, tilpasse til mine ideer og skrive inn i prosjektet selv. Jeg brukte ikke KI til å generere hele prosjektet ferdig, men som hjelp til problemløsning, testing og forbedring underveis.

Egne tanker og refleksjoner

Jeg er vokst opp med KI som en ressurs. Etter korona var kunstig intelligens et blomstrende fenomen som lærere oppfordret oss til å utforske, med delvis retningslinjer for hva det kunne brukes til. Jeg har deltatt på arrangementer hvor jeg har fått mulighet til å snakke med næringsliv som spesifiserer at de ønsker engasjerte nyutdannede som kan og bruke KI, noe jeg ikke tror man kan forstå med mindre man virkelig bruker det i praksis. KI er her for å bli og kommer til å videre effektivisere arbeid.

Samtidig er det utfordrende å vite hvordan man skal forholde seg til KI ettersom at retningslinjene endrer seg fra arena til arena. Jeg jobber med personopplysninger som ikke kan komme ut til eksterne parter, men ettersom vi har tilgang til Co-Pilot360 med beskyttet data kan jeg oppgi så mange personopplysninger jeg vil. Jeg er også skeptisk til dette med tanke på at Microsoft er et amerikansk teknologiselskap i dagens polariserte tilstander kan det jo stilles spørsmålstegn til sikkerheten der også.

Ettersom jeg studerer et IT-relatert studie har jeg lurt på om jeg er “cooked” med tanke på avanserte språkmodellers tilgjengelighet, men til syvende og sist kommer det alltid til å måtte sitte et menneske som kan det som skjer. Det vil ikke være nødvendig, men jeg håper hvert fall at det kommer til å være et krav, slik at ikke verden blir tatt over av kunstig intelligens. Inntil videre er det mange bedrifter som ønsker å omstille interne prosesser og programmer til noe som kan gå av seg selv med kunstig intelligens. Så de fleste bedrifter kommer til å ha bruk for konsulenter som peker de i riktig retning.

KI kan ta over kode-skrivinga, det eneste som trengs å gjøres dersom man skal ha en app, er å lage en god prompt som forklarer KI hva den skal lage. Det samme kan sies om veldig mange andre jobber, hvis man bare er kreativ nok og har nok midler kan nok alle jobber tas over av en robåt til slutt. Jeg håper bare jeg kan få meg en stor pensjon inntil det skjer.